
Matrix Rank Calculator Using C Programming

A Study in Gaussian Elimination, Multidimensional Arrays, and Numerical Rank Determination

Project Title	Matrix Rank Calculator Using C Programming
Subject	Programming for Problem Solving (C)
Project Type	Console-Based Mini Project
Difficulty Level	Beginner
Semester	Semester 1, B.Tech / GSSIPU Curriculum
Author	Mayank Swaraj

Abstract

This paper presents the design, mathematical foundation, and implementation of a console-based Matrix Rank Calculator developed in the C programming language. The rank of a matrix, defined as the maximum number of linearly independent rows or columns, is a fundamental quantity in linear algebra with direct applications in solving systems of linear equations, determining matrix invertibility, and analyzing vector spaces. The implemented program computes matrix rank using an in-place Gaussian elimination procedure operating on a two-dimensional array, incorporating pivot selection, row elimination, and row-swap handling for degenerate (zero-pivot) cases. This paper documents the underlying algorithm, walks through its C implementation, evaluates its time complexity and limitations, and situates the project within the broader pedagogical context of introductory problem-solving courses in computer science curricula such as the GGSIPU B.Tech Semester 1 syllabus.

Keywords: matrix rank, Gaussian elimination, linear algebra, C programming, two-dimensional arrays, row echelon form

1. Introduction

Matrix rank is one of the earliest non-trivial concepts encountered when linear algebra is applied computationally. Formally, for a matrix A of size $m \times n$, the rank is the dimension of the vector space spanned by its rows (row rank), which is provably equal to the dimension of the space spanned by its columns (column rank) — a result known as the rank theorem. Determining rank by hand for small matrices is feasible through row reduction, but doing so programmatically requires translating the row-reduction procedure into an algorithm that a machine can execute deterministically and efficiently.

This project was undertaken as a Semester 1 mini project under the subject Programming for Problem Solving (C), with the goal of reinforcing four core competencies expected at this stage of a computer science curriculum: manipulation of multidimensional arrays, control flow via nested loops and conditionals, translation of a mathematical procedure into imperative code, and disciplined handling of edge cases such as zero pivots during elimination.

2. Mathematical Background

The rank of a matrix can be computed by reducing it to row echelon form through a sequence of elementary row operations: (i) swapping two rows, (ii) multiplying a row by a nonzero scalar, and (iii) adding a scalar multiple of one row to another. None of these operations change the row space of the matrix, and therefore none change its rank. Once the matrix is in row echelon form, the rank equals the number of nonzero rows — equivalently, the number of pivot (leading nonzero) entries.

Gaussian elimination formalizes this reduction. For each pivot column i , if the diagonal entry $A[i][i]$ is nonzero, it is used to eliminate all other entries in that column across every row $j \neq i$ using the update rule shown in Equation 1. If the diagonal entry is zero, the algorithm searches for a row below with a nonzero entry in that column and swaps it into position; if no such row exists, the column contributes no new pivot and the effective rank is decremented.

Equation 1 (row elimination): $A[j][k] \leftarrow A[j][k] - (A[j][i] / A[i][i]) \cdot A[i][k]$, for all k in the active column range, applied whenever $A[j][i]$ must be zeroed relative to pivot row i .

3. Algorithm Design

The algorithm implemented in this project follows a ten-step structure, summarized in Table 1. The central design decision is that rank is initialized optimistically to the number of columns and is decremented only when a column is found to be entirely reducible to zero after exhausting row-swap options — i.e., when that column offers no further linearly independent direction relative to the columns already processed.

1	Start the program.
2	Input the number of rows and columns of the matrix.
3	Input matrix elements from the user.

4	Assume rank equals the number of columns (optimistic initialization).
5	Convert the matrix into row echelon form via elimination.
6	Examine each diagonal (pivot) element in turn.
7	If a diagonal element is zero, attempt a row swap; if none is possible, reduce rank.
8	Continue the elimination process across remaining columns.
9	Display the final computed rank.
10	End the program.

Table 1. Ten-step algorithmic structure of the Matrix Rank Calculator.

4. Implementation in C

The program stores the matrix in a fixed-size two-dimensional array `int matrix[10][10]`, supporting matrices up to 10×10 without dynamic memory allocation — an appropriate constraint for a Semester 1 project where pointers and `malloc` have not yet been introduced. Input is read via nested `scanf` loops, and the elimination logic operates directly on this array using integer indices `i`, `j`, and `k` for the pivot row, target row, and column sweep respectively.

A notable implementation detail is the handling of the zero-pivot case. Rather than terminating early, the program performs a bounded linear search for a nonzero entry below the current pivot row; if found, the rows are swapped via a temporary variable, and elimination proceeds. If no nonzero entry exists in that column below the pivot, the algorithm treats the column as linearly dependent on prior columns: it decrements the rank, shifts the last active column into the position of the exhausted one, and re-examines the same row index (via `i--`) before the loop's increment restores it — effectively shrinking the active column window by one.

5. Complexity Analysis

Time Complexity	$O(\min(m,n) \cdot m \cdot n)$	Each of up to n pivot passes eliminates across m rows and up to n columns.
Space Complexity	$O(m \cdot n)$	In-place elimination on the fixed 10×10 array; no auxiliary matrix allocated.
Numerical Stability	Evidence-supported limitation	No partial pivoting by magnitude is performed; only zero/nonzero pivot swaps, which are limited by the integer range of the input.
Input Domain	Integer entries	Matrix entries are read as <code>int</code> ; the ratio computation is cast to <code>float</code> internally for elimination.

Table 2. Complexity and numerical characteristics of the implementation.

6. Worked Example and Verification

Consider the 3×3 input matrix with rows $(1, 2, 3)$, $(2, 4, 6)$, and $(1, 1, 1)$. The second row is exactly twice the first row, so the two rows are linearly dependent and the row space is spanned by at most two independent directions. Applying the elimination procedure confirms this: after processing the first pivot column, the second

row collapses entirely to zero, while the third row remains independent. The algorithm correctly reports a final rank of 2, matching manual row-reduction verification and confirming the program's correctness on this test case.

Verification status: *Evidence-supported — confirmed by manual Gaussian elimination performed independently of the program's execution trace, and reproduced identically by the interactive HTML companion tool accompanying this paper.*

7. Limitations and Future Work

- The fixed array size (10×10) caps the maximum matrix dimensions; a production-grade version would use dynamic allocation to remove this ceiling.
- Pivoting is selected purely by zero/nonzero status rather than by largest-magnitude entry, which is adequate for integer classroom inputs but can be numerically unstable on floating-point matrices with entries of widely varying scale.
- The program handles only real-valued integer input; extension to rational or complex entries is a natural next step for a Semester 2 follow-up project.
- No unit tests are bundled with the console program; the interactive HTML companion tool serves as an informal correctness check but a formal test harness (e.g., comparing against NumPy's `matrix_rank` on a batch of random matrices) would strengthen validation.

8. Conclusion

This project demonstrates that a foundational linear algebra computation — matrix rank via Gaussian elimination — can be implemented cleanly within the constraints of an introductory C programming course, using only arrays, loops, and conditionals. Beyond its immediate curricular purpose, the project illustrates a pattern common throughout computational mathematics: a well-defined mathematical procedure (row reduction) translates directly into an efficient, deterministic algorithm once its edge cases (zero pivots, row dependency) are enumerated explicitly. The resulting program correctly computes rank for standard test cases and provides a solid foundation for later extensions involving determinants, matrix inverses, and solving linear systems.

References

- Strang, G. *Introduction to Linear Algebra*, 5th Edition. Wellesley-Cambridge Press, 2016.
- Kernighan, B. W., and Ritchie, D. M. *The C Programming Language*, 2nd Edition. Prentice Hall, 1988.
- Golub, G. H., and Van Loan, C. F. *Matrix Computations*, 4th Edition. Johns Hopkins University Press, 2013.
- GGSIPU B.Tech Computer Science Engineering Curriculum, Semester 1 — Programming for Problem Solving (C) course structure.

— Mayank Swaraj